

PSFOM++ Formalization (Fidelity-First) and Implementation Appendix

This paper introduces a Prime-Safe Flow Orchestration Module (PSFOM) that generalizes the crash safety innovations of Arnold Siegel into a robust cognitive safety framework for hybrid quantum-classical AI systems. Drawing from Siegel's work in biomechanical force redirection, child seat engineering, and vehicular crash analysis, we construct a multi-layered system for recursive control, semantic load management, and failure recovery. Assertion boundary and claim scoping

1.1 Defensible core

PSFOM++ treats the following as first-order commitments because they are explicitly stated and/or formally defined:

1. **PSFOM modular safety architecture** including CSB, QTL, snapback recovery (and optional additional layers).
2. **CSB (Cognitive Stopping Buffer)** defined by the PDF equation (see §2.3.1).
3. **QTL entropy gating** with $\backslash S_{\{task\}} = -\sum_k p_k \log p_k$ and threshold behavior.
4. **Snapback recovery** as $\backslash R_{\backslash \psi} = T_{\backslash p}^{-1} \circ \backslash X_i^{-1} \circ \backslash \Pi_{\{truth\}} \backslash . \backslash$

1.2 Conditional capabilities (present in PDF, not enabled as runtime claims)

The following are scoped as conditional/research directions unless and until concrete protocols/hardware/pipelines are specified:

- Monster Group cohomology as an **online/runtime** diagnostic.
- Entanglement/teleportation semantics for flow control.
- Any closed-form theorem relying on the incomplete $\backslash > \sqrt{\backslash \pi \backslash k}$ inequality.

1.3 Identity and credibility correction

The referenced automotive safety engineer is **Arnold (Arne) W. Siegel (1932–2017)** [R1–R4]. Identity resolution uses **domain-context matching** (automotive safety / CPS / SAE-adjacent sources) to avoid conflating unrelated obituary records.

2. PSFOM++ formalization

2.1 Minimal system model

- **State:** $\mathbb{R}^d$ (agent latent + memory + tool state)
- **Task graph:** $G=(V,E)$, finite $|V| < \infty$
- **Depth/scale index:** $\mathbb{N}$; prime indexing uses π_k strictly as a structured depth schedule (not metaphysics).

2.2 Fidelity notes (non-negotiable)

- **Fidelity rule:** PDF equations remain authoritative definitions.
- **Binding rule:** Scalarization/normalization/telemetry mappings are PSFOM++ bindings.
- **Claim rule:** No new proofs asserted; calibration + ablation is used for incomplete or heuristic constraints.

2.3 CSB (Cognitive Stopping Buffer)

2.3.1 Normative definition (PDF)

$$\mathrm{CSB} := \frac{T_p^{(n)} \otimes T_p^{(n+1)}}{2\lambda + T_p \cdot \delta_{\text{latency}}}.$$

2.3.2 PSFOM++ scalar binding (outer product + collapse norm)

- **Binding of $\otimes$:** outer product

$$T_p^{(n)} \otimes T_p^{(n+1)} := \mathrm{einsum}("...i,...j \rightarrow ...ij", T^{(n)}, T^{(n+1)}).$$

- **Collapse norm:** default Frobenius; ablate vs spectral/nuclear.

Scalar CSB binding:

$$\mathrm{CSB}_{\text{raw}} := \frac{|T^{(n)} \otimes T^{(n+1)}|}{2\lambda + \epsilon} + T_p^{\{\mathrm{norm}\}} \cdot \delta_{\text{latency}}^{\{\mathrm{norm}\}}.$$

Note: This binding preserves the *additive structure* of the PDF definition.

2.4 QTL (Quantum Traffic Lights)

2.4.1 Normative definition (PDF)

$$S_{\text{task}} = -\sum_k p_k \log p_k.$$

(Shannon entropy; see [R5].)

2.4.2 PSFOM++ stabilized binding

- - - - Clamp: $p \leftarrow \max(p, \epsilon)$

Gate rule:

- If $S_{task} > S_{max}$: apply backpressure / halt / snapback policy
- Else proceed

2.5 Snapback (recovery)

2.5.1 Normative definition (PDF)

$$R_{\psi} = T_p^{-1} \circ X_i^{-1} \circ \Pi_{\{truth\}}.$$

2.5.2 PSFOM++ enabled mechanism

- **Checkpoint store:** periodic snapshots s_{t_i}
- **Trigger:** CSB violation OR QTL gate OR fracture tier threshold
- **Rollback:** restore s_{t_i}
- **Projection:** apply constraint projection $\Pi_{\{truth\}}$ (truth-manifold proxy)
- **Resume:** continue with tightened CSB/QTL

2.6 Safety condition (repair as calibration; no theorem)

Because the PDF's inequality is incomplete, PSFOM++ uses:

- Margin: $m_k := \text{CSB}_k / (\text{CSB}_{baseline} + \epsilon)$
- Constraint: $m_k \geq \kappa \cdot g(\pi_k)$

Default design prior: sublinear monotone g . Initial $g(\pi_k) = \sqrt{\pi_k}$, compared against $\log(\pi_k)$ and π_k^α .

Coarse grid: $\alpha \in \{0.3, 0.5, 0.7\}$ (optional continuous search after coarse selection).

3. Implementation Appendix (single compact artifact)

3.1 Interfaces

3.1.1 Core data types

- **State:** `State` (opaque; must be serializable for checkpoints)
- **Action logits:** `Tensor[batch, num_routes]`
- **Activations:** `Dict[layer_id, Tensor[... , d_layer]]`

3.1.2 PSFOM runtime interface

Python-like pseudo-interfaces (implementation language agnostic):

- `PSFOMConfig`: configuration schema (§3.2)

- `PSFOMRuntime`: compute metrics, decide gating, perform snapback

PSFOMRuntime

- `init(config: PSFOMConfig, logger: JSONLLogger, clock: Clock)`
- `observe_step_begin(state: State, ctx: StepContext) -> None`
- `observe_logits(logits: Tensor, ctx: StepContext) -> GateDecision`
- `observe_activations(acts: ActivationDict, ctx: StepContext) -> None`
- `observe_step_end(state: State, ctx: StepContext) -> PostStepDecision`
- `maybe_checkpoint(state: State, ctx: StepContext) -> Optional[CheckpointId]`
- `snapback(state: State, ctx: StepContext, reason: Reason) -> State`

3.1.3 Agent wrapper interface

Agent

- `step(state: State) -> (new_state: State, logits: Tensor, acts: ActivationDict, metadata: Dict)`

PSFOMWrappedAgent

- `step(state: State) -> (new_state: State, metadata: Dict)`
 - 1) `runtime.observe_step_begin`
 - 2) `(state', logits, acts, meta) = agent.step(state)`
 - 3) `gate = runtime.observe_logits(logits)`
 - if `gate.halt`: `state' = runtime.snapback(...)`
 - 4) `runtime.observe_activations(acts)`
 - 5) `post = runtime.observe_step_end(state')`
 - if `post.snapback`: `state' = runtime.snapback(...)`
 - 6) `runtime.maybe_checkpoint`
 - 7) `logger.emit(JSONL event)`

3.2 Configuration schema

3.2.1 Canonical schema (YAML or JSON)

```
psfom:
  version: "psfompp-1.0"

# Numerics
eps: 1.0e-8

# CSB bindings
csb:
  collapse_norm: "fro"           # {fro, spectral, nuclear}
  outer_product: true           # defines ⊗ via einsum
  momentum_ema_beta: 0.9
  xi_window_w: 10
```

```

latency_baseline_window: 50

# QTL bindings
qtl:
  tau: 1.0
  s_max_policy: "quantile"      # {fixed, quantile}
  s_max_fixed: 2.5             # used if fixed
  s_max_quantile: 0.95         # used if quantile

# Safety calibration
safety:
  kappa: 1.0
  g_family: "sqrt"             # {sqrt, log, power}
  alpha: 0.5                   # used if power
  fp_target: 0.05

# Checkpoint & recovery
recovery:
  checkpoint_interval_steps: 25
  max_rollback_steps: 200
  projection: "constraints_v1"  # implementation-defined

# Diagnostics
diagnostics:
  fracture_tier_enabled: 2      # 0..3 runtime; 4 offline only

# Experiments
experiments:
  seeds: [0,1,2,3,4]
  traces_per_seed: 100
  max_steps_per_trace: 256

# Ablations
ablations:
  index_schemes: ["prime","linear","fibonacci","learned"]
  g_families: ["sqrt","log","power"]
  alphas: [0.3,0.5,0.7]
  csb_norms: ["fro","spectral"]
  taus: [0.7,1.0,1.3]
  xi_windows: [5,10,20]
  kappa_perturb: [0.9,1.0,1.1]

```

3.3 Agent wrapper pseudocode

```

class PSFOMWrapperAgent:
    def __init__(self, agent, runtime, logger):

```

```

        self.agent = agent
        self.runtime = runtime
        self.logger = logger

    def step(self, state, ctx):
        self.runtime.observe_step_begin(state, ctx)

        # Execute base agent step
        new_state, logits, acts, meta = self.agent.step(state)

        # QTL gating
        gate = self.runtime.observe_logits(logits, ctx)
        if gate.halt:
            new_state = self.runtime.snapback(new_state, ctx,
reason=gate.reason)
            meta["psfom_gate"] = gate.to_dict()

        # CSB observables
        self.runtime.observe_activations(acts, ctx)

        # Post-step evaluation (CSB violations, fracture tiers, etc.)
        post = self.runtime.observe_step_end(new_state, ctx)
        if post.snapback:
            new_state = self.runtime.snapback(new_state, ctx,
reason=post.reason)
            meta["psfom_post"] = post.to_dict()

        # Checkpoint
        ckpt_id = self.runtime.maybe_checkpoint(new_state, ctx)
        meta["checkpoint_id"] = ckpt_id

        # Emit JSONL log
        event = self.runtime.build_event(ctx, meta)
        self.logger.emit(event)

    return new_state, meta

```

3.4 Minimal harness (Phase 1/2/3 runner)

3.4.1 Harness responsibilities

- Load config
- Instantiate base agent + PSFOM runtime
- Run traces with stressors
- Emit JSONL logs
- Compute metrics + deltas vs baseline
- Run Phase 2 sensitivities + Phase 3 graph scheduling tests

3.4.2 Harness pseudocode

```
def run_harness(config):
    # 0) Prepare output
    out_dir = make_run_dir(config)
    logger = JSONLLogger(path=out_dir/"events.jsonl")

    # 1) Build agents
    base_agent = build_agent(config)
    psfom_runtime = PSFOMRuntime(config, logger)
    wrapped_agent = PSFOMWrappedAgent(base_agent, psfom_runtime, logger)

    # 2) Define baseline runner (no PSFOM, or fixed depth cap only)
    baseline_runner = BaselineRunner(base_agent, config, logger=None)

    # 3) Phase 1: core safety loop
    results_baseline = run_traces(baseline_runner, config,
    stressors=core_stressors())
    results_psfom = run_traces(wrapped_agent, config,
    stressors=core_stressors())
    report_phase1(out_dir, results_baseline, results_psfom)

    # 4) Phase 2: sensitivity + norm-choice ablation
    for variant in iter_phase2_variants(config):
        psfom_runtime = PSFOMRuntime(variant, logger)
        wrapped = PSFOMWrappedAgent(base_agent, psfom_runtime, logger)
        res = run_traces(wrapped, variant, stressors=core_stressors())
        save_variant_metrics(out_dir, variant, res)
    report_phase2(out_dir)

    # 5) Phase 3: scheduling/tiling scoped to DAGs
    for sched_case in build_dag_benchmarks(config):
        res_sched = run_graph_scheduling_eval(sched_case, config)
        save_sched_metrics(out_dir, sched_case, res_sched)
    report_phase3(out_dir)

    return out_dir
```

3.4.3 Stressors (minimum set)

- **Contradiction injection:** introduce structured inconsistencies into planning context.
- **Circuit-breaker stress:** simulate repeated tool failures/timeouts to test “fail-fast” behavior and exponential backoff under cascading errors (circuit-breaker lineage; see [R10]).
- **Branching overload:** force large route sets / tool-call fan-out.
- **Latency perturbation:** simulate slow tools or network delay.

3.5 JSONL event format (canonical)

(JSON Lines / newline-delimited JSON; see [R20].) One event per step:

```
{
  "run_id": "...",
  "seed": 0,
  "git_commit": "...",
  "ts_ms": 0,
  "step": 0,
  "k": 0,
  "pi_k": 2,
  "ablation_meta": {
    "index_scheme": "prime",
    "g_family": "sqrt",
    "alpha": 0.5,
    "csb_norm": "fro",
    "tau": 1.0,
    "w": 10,
    "beta": 0.9
  },
  "metrics": {
    "csb_raw": 0.0,
    "csb_baseline": 0.0,
    "m_k": 0.0,
    "Xi": 0.0,
    "Tp": 0.0,
    "dlat": 0.0,
    "S_task": 0.0
  },
  "gate": {
    "qtl": false,
    "csb_violation": false,
    "fracture_tier": 0
  },
  "recovery": {
    "did_snapback": false,
    "checkpoint_id": null,
    "projection": "constraints_v1",
    "recovery_steps": 0
  }
}
```


3.6 Experiment definitions (Phase 1/2/3)

Phase 1: Core safety loop

- Compare baseline vs PSFOM++ on identical seeds/tasks.
- **Primary metrics (relative):** runaway depth events, mean recovery time, overhead, task success.

Phase 2: Sensitivity + ablations (fixed protocol)

Objective: quantify robustness and isolate which PSFOM++ bindings materially drive stability/overhead tradeoffs, without conflating calibration with true effect.

2.1 Experimental design (required)

1. **Freeze the task/stressor suite** used in Phase 1 (same generator + same seeds).
2. **Separate calibration from evaluation** (to prevent “tuning on the test”):
3. **Calibration split:** first 30% of traces per seed.
4. **Evaluation split:** remaining 70% of traces per seed.
5. All Phase 2 comparisons are reported on the **evaluation split only**.
6. **Baseline controls (always include):**
7. `baseline/none`: no PSFOM.
8. `baseline/cap`: fixed recursion depth cap (matched average depth to PSFOM run).
9. `psfom/default`: v1.2 defaults.

2.2 Parameters and ablation axes (v1.2)

A. Sensitivity (local robustness around the chosen operating point)

- $\kappa \in \{0.9, 1.0, 1.1\}$ ($\pm 10\%$)
- $\tau \in \{0.7, 1.0, 1.3\}$
- $w \in \{5, 10, 20\}$ for $\exists \Delta$ window
- CSB collapse norm: {fro, spectral} (add nuclear only if stable/feasible)

B. Structural ablations (hypothesis tests)

- **Index scheme:** {prime, linear, fibonacci, learned_monotone}
- **Margin family:** $g \in \{\text{sqrt}, \text{log}, \text{power}\}$ with $\alpha \in \{0.3, 0.5, 0.7\}$ for `power`
- **QTL thresholding policy:** $S_{\text{max_policy}} \in \{\text{fixed}, \text{quantile}\}$
- fixed: $S_{\text{max_fixed}} \in \{2.0, 2.5, 3.0\}$ (nats)
- quantile: $S_{\text{max_quantile}} \in \{0.90, 0.95, 0.98\}$

C. Learned indexing constraint (to avoid overfit and non-monotone artifacts)

- Require a **monotone mapping** from depth k (or π_k) to scheduling scale.
- Default implementation: **isotonic regression** fit on the calibration split, evaluated on the evaluation split.

2.3 Calibration procedure (what may be fit, and when)

Only the following may be fit, and only on the calibration split:

- κ (to hit target false-positive rate `fp_target` on “safe” traces)
- If `S_max_policy=quantile`: the rolling quantile estimator state (windowed)

Explicitly not fit in Phase 2:

- index scheme, norm choice, g family, α , τ , w (these are compared as ablations)

2.4 Stationarity handling for quantile gating (required if used)

If `S_max_policy=quantile`, compute `S_max` with a **rolling window** over the last N steps (default N=200) and log the window stats. This prevents a single early spike from setting `S_max` for the entire trace.

2.5 Reporting (minimum)

For each variant (including baselines), report on the evaluation split:

- Stability: runaway events / 1k steps; snapback rate; contradiction rate; cycle incidence (if applicable)
- Recovery: mean/median recovery time; P95 recovery time
- Overhead: latency delta; compute delta; OOM/shape-error count
- Task success: completion rate; normalized score (task-specific)

Effect sizes (required):

- Cohen's d (or Cliff's delta for non-normal metrics) vs `baseline/cap`.

Selection rule (required):

- choose configurations on the **Pareto frontier** (stability vs overhead), then pick one operating point per index scheme for Phase 3.

2.6 Logging requirements (Phase 2)

Every JSONL event must include:

- `ablation_meta` (index scheme, g family, alpha, norm, tau, w, kappa)
- `error` populated on any runtime issue
- a per-run `config_hash` (in run metadata) so variants are unambiguous

Phase 3: Graph scheduling (scoped)

- **Guarantee scope**: DAG task graphs only (max-flow / flow-conservation tools apply cleanly in this restricted case; see [R8], with common implementations in [R12–R13]).
- Cycles: heuristic cycle detection + CSB/QTL backpressure (no general deadlock guarantee).

3.7 Reproducibility clause

All code, configurations, seeds, and JSONL traces required to reproduce results must be released with:

- fixed seeds for ablation comparability,
 - a versioned configuration hash per run,
 - run directory containing: `config.yaml`, `events.jsonl`, `metrics.json`, `summary.md`.
-

4. Implementation checklist (for an MVP)

1. Implement `PSFOMRuntime` computations: $\backslash X_i \backslash \Lambda$, $\backslash T_p \backslash$, $\backslash \Delta \backslash \{latency\}$, CSB scalar binding, QTL entropy.
 2. Implement checkpoint store + snapback projection adapter.
 3. Implement baseline runner + wrapped agent.
 4. Emit JSONL at every step with required fields.
 5. Implement metrics aggregation: runaway depth events, recovery time, overhead, task success.
 6. Implement Phase 2 variant iterator and reporting.
 7. Implement Phase 3 DAG scheduling evaluator and reporting.
-

5. Notes on future extensions (non-blocking)

- Tier 3 topology proxies for fracture diagnostics (persistent homology / computational topology; see [R16]).
 - Offline Monster-inspired diagnostics (Moonshine context; see [R17–R19]).
 - Quantum hardware integration for QTL (conditional extension only).
 - Aperiodic tiling / quasilattice scheduling heuristics (mathematical and physical context; see [R14–R15]).
-

6. External references

[R1] Motorcycle Accident Cause Prevention System (MACPS), *Child Passenger Safety Hall of Fame: Arnold W. Siegel (1932–2017)*, organizational PDF hosted by Safe Ride News (accessed 2025-12-28).

[R2] SAE International, *Arnold W. Siegel International Transportation Safety Award* (award description and history; accessed 2025-12-28).

[R3] SAE International, *Abelson Award for Visionary Leadership* (past recipients list; accessed 2025-12-28).

[R4] Legacy.com (Los Angeles Times), *Arnold Siegel Obituary* (2017; accessed 2025-12-28).

[R5] C. E. Shannon, “A Mathematical Theory of Communication,” *The Bell System Technical Journal*, vol. 27, pp. 379–423 and 623–656, 1948.

- [R6]** S. Banach, “Sur les opérations dans les ensembles abstraits et leur application aux équations intégrales,” *Fundamenta Mathematicae*, 1922. (Classical contraction/fixed-point principle provenance.)
- [R7]** T. M. Apostol, *Introduction to Analytic Number Theory*, Springer, 1976. DOI: 10.1007/978-1-4757-5579-4.
- [R8]** L. R. Ford Jr. and D. R. Fulkerson, “Maximal Flow Through a Network,” *Canadian Journal of Mathematics*, 8:399–404, 1956. DOI: 10.4153/CJM-1956-045-5.
- [R9]** K. M. Chandy and L. Lamport, “Distributed Snapshots: Determining Global States of a Distributed System,” *ACM Transactions on Computer Systems*, 3(1):63–75, 1985. DOI: 10.1145/214451.214456.
- [R10]** M. T. Nygard, *Release It! (Second Edition): Design and Deploy Production-Ready Software*, Pragmatic Bookshelf, 2018; see also M. Fowler, “Circuit Breaker” (pattern note). (Cascading-failure control.)
- [R11]** L. Tassiulas and A. Ephremides, “Stability Properties of Constrained Queueing Systems and Scheduling Policies for Maximum Throughput in Multihop Radio Networks,” *IEEE Transactions on Automatic Control*, 37(12):1936–1948, 1992. DOI: 10.1109/9.182479.
- [R12]** A. A. Hagberg, D. A. Schult, and P. J. Swart, “Exploring Network Structure, Dynamics, and Function using NetworkX,” in *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, 2008.
- [R13]** NetworkX Documentation, `networkx.algorithms.flow` module (maximum-flow routines; accessed 2025-12-28).
- [R14]** R. Penrose, “The role of aesthetics in pure and applied mathematical research,” *Bulletin of the Institute of Mathematics and its Applications*, 10:266–271, 1974. (Aperiodic tiling context.)
- [R15]** D. Shechtman, I. Blech, D. Gratias, and J. W. Cahn, “Metallic Phase with Long-Range Orientational Order and No Translational Symmetry,” *Physical Review Letters*, 53:1951–1953, 1984. DOI: 10.1103/PhysRevLett.53.1951.
- [R16]** H. Edelsbrunner and J. Harer, *Computational Topology: An Introduction*, American Mathematical Society, 2010.
- [R17]** J. H. Conway and S. P. Norton, “Monstrous Moonshine,” *Bulletin of the London Mathematical Society*, 11:308–339, 1979. DOI: 10.1112/blms/11.3.308.
- [R18]** R. E. Borcherds, “Monstrous Moonshine and Monstrous Lie Superalgebras,” *Inventiones Mathematicae*, 109:405–444, 1992. DOI: 10.1007/BF01233420.
- [R19]** T. Gannon, *Moonshine Beyond the Monster: The Bridge Connecting Algebra, Modular Forms and Physics*, Cambridge University Press, 2006.
- [R20]** JSON Lines / newline-delimited JSON format documentation (jsonlines; accessed 2025-12-28).

Reference URLs (verification)

R1: saferidenews.com/macps/cps-hof/
R2: sae.org/participate/awards/arnold-w-siegel-international-transportation-safety-award
R3: sae.org/participate/awards/abelson-award
R4: legacy.com/us/obituaries/latimes/name/arnold-siegel-obituary
R5: people.math.harvard.edu/~ctm/home/text/others/shannon/entropy/entropy.pdf
R6: eudml.org/doc/213289
R7: link.springer.com/book/10.1007/978-1-4757-5579-4
R8: doi.org/10.4153/cjm-1956-045-5
R9: dl.acm.org/doi/10.1145/214451.214456
R10: pragprog.com/titles/mnee2/release-it-second-edition/ ; martinfowler.com/bliki/CircuitBreaker.html
R11: doi.org/10.1109/9.182479
R12: conference.scipy.org/proceedings/scipy2008/paper_2/
R13: networkx.org/documentation/stable/reference/algorithms.flow.html
R14: (paper metadata varies by archive; search "Penrose 1974 aesthetics tilings IMA Bulletin")
R15: doi.org/10.1103/PhysRevLett.53.1951
R16: ams.org/books/stml/069/
R17: doi.org/10.1112/blms/11.3.308
R18: doi.org/10.1007/BF01233420
R19: cambridge.org/core/books/moonshine-beyond-the-monster/
R20: jsonlines.readthedocs.io/